Análise Comparativa de Estratégias de Controle Populacional em Algoritmos Genéticos aplicados ao Problema das N-Rainhas

Vinícius Rubin[1], **Igor Yepes**[1]

[1]Instituto de Educação, Ciência e Tecnologia Farroupilha - Campus Frederico Westphalen (IFFAR-FW)

Abstract. *This article investigates how different population control strategies affect the performance of genetic algorithms when solving the n-queens problem. We describe the design of a baseline implementation, outline the experimental protocol, and discuss how metrics such as execution time, number of generations, and success rate will be interpreted. The study aims to identify configurations that preserve diversity while accelerating convergence, contributing guidelines for practitioners who deploy evolutionary heuristics in combinatorial search tasks.*

Resumo. *Este artigo investiga como diferentes estratégias de controle populacional influenciam o desempenho de algoritmos genéticos na resolução do problema das n-rainhas. São descritos o planejamento da implementação base, o protocolo experimental e a forma de análise de métricas como tempo de execução, número de gerações e taxa de sucesso. O estudo busca apontar configurações que preservem a diversidade e acelerem a convergência, oferecendo subsídios para o emprego de heurísticas evolutivas em tarefas de busca combinatória.*

1. Introdução

Algoritmos Genéticos (AGs) são técnicas de otimização baseadas nos princípios da seleção natural e da evolução biológica, onde soluções potenciais são representadas por indivíduos que evoluem ao longo de gerações por meio de operadores como seleção, cruzamento e mutação [Holland 1975]. Devido à sua capacidade de explorar grandes espaços de busca e lidar com problemas complexos e não lineares, os AGs têm sido amplamente aplicados em áreas como engenharia, logística, inteligência artificial e bioinformática.

Uma das principais vantagens dos AGs está em sua flexibilidade: com poucos ajustes nos operadores ou nos parâmetros, é possível adaptá-los a diferentes domínios de aplicação. No entanto, essa mesma flexibilidade impõe desafios, já que o desempenho do algoritmo depende diretamente das escolhas realizadas em sua configuração. Entre essas escolhas, destaca-se o controle populacional, que define como a população é formada e evolui a cada geração — influenciando diretamente a diversidade genética e a qualidade das soluções encontradas [Eiben and Smith 2015, de Lacerda and de Leon Ferreira de Carvalho 1999].

Apesar da existência de diversas estratégias de controle populacional, como elitismo, substituição parcial ou total, ainda há escassez de estudos que comparem sistematicamente os impactos dessas variações em um mesmo cenário experimental. A falta de

padronização nos testes, aliada à diversidade dos problemas abordados, dificulta a análise objetiva do comportamento dessas estratégias e de seus efeitos sobre o desempenho dos AGs.

Neste contexto, este trabalho propõe uma investigação sobre o impacto de diferentes técnicas de controle populacional no desempenho de Algoritmos Genéticos (AGs). Para isso, será utilizado o problema das n-rainhas como ambiente de teste. Esse problema clássico da computação combinatória consiste em posicionar n rainhas em um tabuleiro de $n \times n$ casas de modo que nenhuma delas possa atacar outra. Derivado diretamente do jogo de xadrez, que é reconhecido por sua profundidade estratégica e complexidade computacional, o problema das n-rainhas destaca-se por sua estrutura bem definida, escalabilidade controlada e facilidade de replicação, tornando-se um benchmark amplamente adotado em experimentos com algoritmos de busca e otimização.

A proposta central é desenvolver uma implementação base de AG em Python e aplicar diferentes variações de controle populacional — substituição total, substituição parcial e elitismo — mantendo constantes os demais operadores (seleção, cruzamento e mutação). Para cada variação, serão realizados experimentos repetidos com diferentes valores de n, coletando métricas como tempo de execução, número de gerações até a convergência e taxa de sucesso.

Os resultados obtidos serão avaliados a partir de um conjunto amplo de estatísticas descritivas e inferenciais calculadas sobre 100 execuções independentes por configuração. Além das médias de desempenho, serão reportados os desvios padrão, intervalos de confiança de 95% e tamanhos de efeito. Testes de significância auxiliarão na comparação formal entre as estratégias de controle populacional, fornecendo evidências quantitativas sobre eventuais diferenças observadas. Com esse arranjo experimental, espera-se identificar quais abordagens oferecem ganhos concretos de desempenho e fornecer subsídios metodológicos para a construção de AGs mais eficientes em problemas de natureza semelhante.

2. Fundamentação Teórica

2.1. Algoritmos Genéticos

Algoritmos Genéticos (AGs) são técnicas de busca e otimização baseadas nos princípios da evolução natural, como seleção, cruzamento e mutação. Propostos inicialmente por Holland na década de 1970 [Holland 1975], esses algoritmos operam sobre uma população de soluções candidatas — os indivíduos — que evoluem ao longo de várias gerações em direção a soluções mais adaptadas.

Cada indivíduo é codificado de acordo com a estrutura do problema em questão e avaliado por uma função de aptidão (*fitness*), que determina sua qualidade. Segundo Lacerda e Carvalho [de Lacerda and de Leon Ferreira de Carvalho 1999], o desempenho dos AGs depende diretamente da escolha da codificação, da função de avaliação e dos operadores genéticos utilizados.

O funcionamento de um algoritmo genético se baseia em um ciclo evolutivo composto por etapas bem definidas, que se repetem a cada geração. Essas etapas incluem:

- **Inicialização**: geração aleatória da população inicial, o que assegura diversidade e ampla cobertura do espaço de busca;

- **Avaliação**: cálculo da aptidão de cada indivíduo com base nos critérios definidos para o problema;
- **Seleção**: escolha dos indivíduos mais aptos para reprodução;
- **Cruzamento**: combinação de pares de indivíduos para gerar descendentes;
- **Mutação**: introdução de pequenas alterações aleatórias nos indivíduos, essencial para manter a diversidade genética [Spears 1993];
- **Substituição**: definição de quais indivíduos permanecem na nova geração, com abordagens como elitismo, substituição total ou parcial.

A Figura 1 ilustra esse ciclo evolutivo, evidenciando o fluxo entre as etapas principais do algoritmo genético.

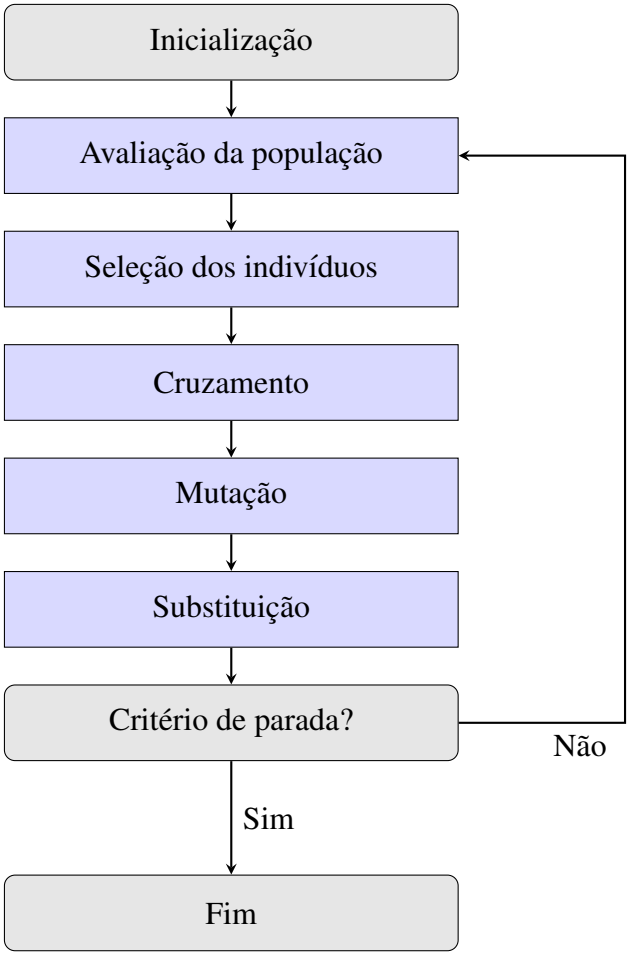


Figura 1. Fluxograma do funcionamento do algoritmo genético.

2.2. O Problema das *n*-Rainhas

O problema das n-rainhas consiste em posicionar n rainhas em um tabuleiro de xadrez de dimensões $n \times n$ de forma que nenhuma delas possa atacar outra. Isso significa que não pode haver mais de uma rainha em qualquer linha, coluna ou diagonal [Bell and Stevens 2009]. Proposto originalmente em 1848 por Max Bezzel e posteriormente analisado por matemáticos como Gauss e Nauck, esse problema possui uma longa tradição tanto na matemática recreativa quanto na ciência da computação [Bell and Stevens 2009].

Apesar de sua formulação simples, o número de possíveis configurações cresce exponencialmente com o aumento de n, tornando abordagens exaustivas inviáveis para valores elevados. Conforme demonstrado por Ali et al. [Ali et al. 2023], algoritmos clássicos como busca em largura (BFS) e busca em profundidade (DFS) são eficazes para instâncias pequenas, mas tornam-se computacionalmente custosos à medida que o tabuleiro cresce. A escalabilidade do problema, aliada à sua estrutura bem definida e múltiplas soluções conhecidas, faz dele um excelente benchmark para avaliar o desempenho de algoritmos de busca e otimização.

Por essas razões, o problema das n-rainhas tem sido amplamente adotado como ambiente de teste para algoritmos evolutivos e de busca heurística [Sarkar and Nag 2017, Farhan et al. 2015, Ali et al. 2023]. Sua natureza combinatória, aliada à clareza das restrições e à escalabilidade controlada, permite uma investigação aprofundada de estratégias de codificação, operadores genéticos e mecanismos de seleção em um contexto clássico e amplamente reconhecido na literatura.

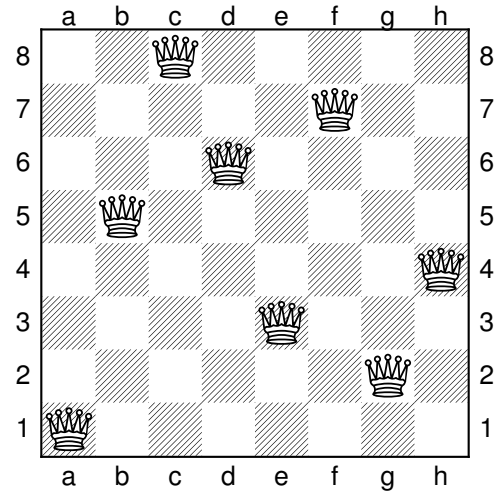


Figura 2. Exemplo de solução válida para o problema das 8-rainhas.

2.3. Uso da Linguagem Python e Bibliotecas Relacionadas

A linguagem Python será adotada neste projeto para a implementação e execução dos experimentos com algoritmos genéticos. Sua escolha se justifica pela clareza sintática, ampla aceitação na comunidade científica e suporte a bibliotecas voltadas à análise de dados e visualização [Python Software Foundation 2025a].

Para gerar variabilidade genética e aplicar operadores estocásticos, será utilizado o módulo `random`, que oferece ferramentas para geração de números pseudoaleatórios, amostragem e embaralhamento de sequências [Python Software Foundation 2025b].

A manipulação de vetores e matrizes será realizada com a biblioteca `NumPy`, que fornece estruturas otimizadas e operações vetorizadas de alto desempenho [Harris et al. 2020]. Essa ferramenta será fundamental para representar populações, aplicar operadores genéticos em escala e calcular a função de aptidão com eficiência.

A cronometragem dos experimentos será feita com o módulo `time`, que permite

medir o tempo de execução de forma precisa [Python Software Foundation 2025c]. Já a análise visual dos resultados será conduzida com a biblioteca `matplotlib`, que oferece suporte à geração de gráficos científicos e facilita a interpretação das métricas obtidas [Hunter 2007].

Esse conjunto de ferramentas proporcionará uma base robusta e reprodutível para o desenvolvimento do projeto e a apresentação dos resultados.

2.4. Trabalhos Relacionados

Diversos estudos têm investigado o impacto de variações nos operadores dos algoritmos genéticos (AGs) sobre o desempenho em problemas de otimização combinatória. A escolha da representação dos indivíduos, a estratégia de seleção, a taxa de mutação, bem como a forma de controle populacional, pode influenciar significativamente a eficácia do algoritmo.

2.4.1. Representações Binária e Inteira para o Problema das *n*-Rainhas [Richter and Berlezi 2012]

Richter et al. [Richter and Berlezi 2012] investigaram a aplicação de algoritmos genéticos (AGs) para resolver o problema das *n*-rainhas, comparando duas abordagens distintas de representação dos indivíduos: uma baseada em codificação binária e outra utilizando números inteiros, denominada *InOrder*. O estudo teve como objetivo desenvolver um framework adaptável para problemas similares, como o posicionamento de antenas, utilizando o problema das *n*-rainhas como base experimental devido ao seu caráter combinatório exponencial.

A implementação binária utiliza sequências de bits que representam a posição das rainhas, exigindo conversão para valores decimais e sendo suscetível a colisões horizontais. Já a abordagem *InOrder* codifica diretamente a posição das rainhas por meio de números inteiros únicos em cada indivíduo, evitando conflitos de linha e coluna desde a geração inicial da população. Essa característica garantiu desempenho significativamente superior, especialmente em problemas com maior número de rainhas, como demonstrado nos testes com 16 posições.

Os autores destacam que, apesar de ambas as representações seguirem os princípios clássicos da meta-heurística evolutiva — incluindo seleção por torneio, cruzamento e mutação —, a *InOrder* se mostrou mais eficaz ao evitar soluções inviáveis desde o início do processo evolutivo, reduzindo o número de gerações necessárias para encontrar soluções válidas.

O presente projeto se alinha à proposta de Richter et al. [Richter and Berlezi 2012] ao utilizar o problema das *n*-rainhas como instrumento de análise, mas avança ao comparar diferentes estratégias de controle populacional em algoritmos genéticos. Enquanto o trabalho citado foca na representação genética, esta pesquisa busca compreender o impacto de mecanismos como elitismo, substituição parcial e substituição total, ampliando assim o entendimento sobre a eficácia dos AGs em contextos de otimização combinatória.

2.4.2. Comparação entre Algoritmo Genético e Buscas Clássicas (BFS/DFS) [Ali et al. 2023]

Ali et al. [Ali et al. 2023] realizaram um estudo comparativo entre algoritmos genéticos e técnicas clássicas de busca — busca em largura (BFS) e busca em profundidade (DFS) — aplicadas ao problema das *n*-rainhas. O objetivo foi avaliar a eficácia desses métodos em diferentes tamanhos de tabuleiro e identificar os contextos nos quais os AGs apresentam vantagens reais sobre métodos tradicionais.

O algoritmo genético proposto utiliza uma representação vetorial, onde cada cromossomo é uma string de tamanho N, em que cada posição representa a linha de uma rainha na respectiva coluna. A avaliação da aptidão baseia-se no número de pares de rainhas não conflitantes, sendo o valor ótimo aquele correspondente à combinação total de pares sem ataques. Foram utilizados operadores de seleção por torneio, cruzamento uniforme e mutação por redefinição aleatória, com taxa de mutação de 10%.

Nos experimentos realizados, os autores observaram que para instâncias pequenas (até 8 rainhas), BFS e DFS superaram o algoritmo genético em tempo de execução. No entanto, para problemas médios e grandes (acima de 10 rainhas), o AG apresentou melhor escalabilidade, com crescimento linear no tempo de execução, enquanto os métodos tradicionais exibiram crescimento exponencial.

O estudo também evidencia a sensibilidade do desempenho do AG à configuração de seus hiperparâmetros, como a taxa de mutação. A aleatoriedade natural do algoritmo pode resultar em tempos de convergência variáveis, mas ajustes apropriados garantem sua eficiência.

Este trabalho reforça o potencial dos AGs como alternativa viável em problemas de otimização combinatória de alta complexidade, especialmente em contextos onde técnicas exaustivas se tornam inviáveis. A presente pesquisa se inspira nesse estudo ao adotar o problema das *n*-rainhas como banco de testes, focando na análise de estratégias de controle populacional em AGs.

2.4.3. Algoritmo Genético Adaptativo com Controle de Diversidade [Sarkar and Nag 2017]

Sarkar e Nag [Sarkar and Nag 2017] propuseram uma abordagem adaptativa baseada em algoritmos genéticos para resolver o problema das *n*-rainhas, com melhorias na função de aptidão e mecanismos de controle da diversidade populacional. O trabalho combina operadores clássicos com estratégias inovadoras para evitar o aprisionamento em ótimos locais — um desafio frequente em problemas combinatórios.

A representação dos indivíduos é feita por permutações dos números de 1 a N, garantindo que cada rainha ocupe uma coluna distinta. A função de aptidão penaliza conflitos diagonais, utilizando duas varreduras para detecção de ataques. Indivíduos com valor de aptidão igual a zero representam soluções válidas.

O operador de cruzamento adotado é o *Order 1 Crossover*, apropriado para permutações. A mutação consiste em trocas aleatórias de dois genes, com alta probabilidade (0.8), e uma variação adicional chamada *mutação dupla*, com probabilidade de

0.4. Destaca-se ainda a introdução de um *repositório de má população*, que armazena indivíduos menos aptos para manter a diversidade e evitar estagnações.

Os resultados experimentais mostram que a abordagem encontra soluções ótimas para valores elevados de N (até 100) com eficiência, sugerindo robustez da proposta. Os autores indicam ainda o potencial da metodologia para problemas reais de escalonamento.

Este trabalho é relevante para a presente pesquisa por inspirar mecanismos internos de controle da diversidade e da pressão seletiva — elementos fundamentais para a eficiência de AGs em problemas complexos como o das *n*-rainhas.

2.4.4. Busca de Todas as Soluções Válidas para as 8-Rainhas [Farhan et al. 2015]

Farhan et al. [Farhan et al. 2015] propuseram um algoritmo genético focado em encontrar as 92 soluções válidas do problema das 8 rainhas. O destaque da proposta está na representação vetorial unidimensional, onde o índice do vetor representa a linha e o valor armazenado representa a coluna de cada rainha. Essa representação elimina conflitos em linhas e colunas, restringindo a função de aptidão à verificação de ataques diagonais.

A função de aptidão contabiliza o número de pares de rainhas que se atacam diagonalmente. Um valor zero indica uma solução válida. A estrutura do AG segue os princípios tradicionais: geração de população inicial aleatória, seleção por torneio de três vias, cruzamento (incluindo o operador PMX) e mutação simples por troca de posições.

O algoritmo foi executado até que todas as 92 soluções únicas fossem encontradas. Os resultados demonstram que o AG é capaz de explorar eficientemente o espaço de busca, embora a aleatoriedade influencie o número de iterações necessárias para encontrar cada solução.

Este trabalho é pertinente ao presente estudo por enfatizar a cobertura completa do espaço de soluções e o uso de operadores eficazes. Além disso, reforça a capacidade dos AGs de encontrar múltiplas soluções sem a necessidade de estruturas complexas, servindo como referência para o desenvolvimento de abordagens evolutivas mais avançadas.

3. Metodologia

Este trabalho adota uma abordagem experimental com o objetivo de investigar o impacto de diferentes estratégias de controle populacional em algoritmos genéticos (AGs), aplicados à resolução do problema das *n*-rainhas. A condução metodológica segue as diretrizes propostas por Eiben e Smith [Eiben and Smith 2015] e Goldberg [Goldberg 1989], priorizando a análise quantitativa de desempenho em cenários controlados e reprodutíveis.

3.1. Definição do Algoritmo Genético Base

Com base no referencial teórico, o algoritmo genético utilizado neste estudo segue uma estrutura padrão composta por etapas fundamentais que guiam a evolução das soluções ao longo das gerações. A seguir, são descritas as decisões metodológicas adotadas em cada uma dessas etapas.

3.1.1. Representação do Indivíduo

Cada indivíduo da população é representado por um vetor de permutação de tamanho n, em que a posição do vetor corresponde a uma coluna do tabuleiro e o valor armazenado nessa posição indica a linha ocupada pela rainha. Essa representação elimina conflitos em linhas e colunas, restando apenas os conflitos diagonais a serem tratados na avaliação da aptidão.

3.1.2. Inicialização

A população inicial é composta por indivíduos gerados aleatoriamente. Cada vetor é construído embaralhando uma sequência de inteiros de 0 a $n-1$, garantindo diversidade genética desde o início da evolução. O tamanho da população é definido de acordo com a técnica de controle populacional aplicada em cada experimento.

3.1.3. Avaliação

A função de aptidão (*fitness*) tem como objetivo contar a quantidade de conflitos diagonais existentes em uma configuração. Para isso, todos os pares de rainhas são comparados verificando se $|i - j| = |v[i] - v[j]|$, onde i e j representam índices de colunas distintas e $v[i]$ e $v[j]$ as respectivas linhas ocupadas. A complexidade computacional dessa etapa é de ordem $O(n^2)$, considerada apropriada para os valores de n utilizados neste estudo.

3.1.4. Seleção

Será adotada a estratégia de torneio binário como método de seleção. Em cada torneio, dois indivíduos são escolhidos aleatoriamente da população, e aquele com melhor desempenho (menor número de conflitos) é selecionado para reprodução. O processo é repetido até formar a quantidade necessária de candidatos ao cruzamento, conforme a configuração da geração.

3.1.5. Cruzamento

O operador de cruzamento empregado será o *Order Crossover* (OX), apropriado para problemas de permutação. O procedimento consiste em:

1. Selecionar dois pontos de corte aleatórios no vetor de um dos pais;
2. Copiar a subsequência entre os pontos de corte para o descendente;
3. Preencher as posições restantes com os elementos do outro pai, respeitando a ordem relativa e evitando repetições.

Esse operador garante a validade da solução gerada, mantendo a integridade da permutação e evitando conflitos estruturais.

3.1.6. Mutação

A mutação será realizada por meio da troca aleatória entre duas posições distintas do vetor do indivíduo. Essa operação simples é suficiente para introduzir variabilidade genética ao longo das gerações.

3.1.7. Substituição

Serão testadas três estratégias de substituição para controle populacional, cada uma combinando variações de tamanho populacional com ajustes de taxa de mutação e tamanho de torneio:

- **Substituição total**: toda a população é substituída pelos indivíduos gerados na iteração atual. Cada variação de tamanho populacional (100, 200 e 300 indivíduos) é avaliada com taxas de mutação de 10%, 20% e 30%, combinadas com torneios binários de tamanho $k = 2$;
- **Substituição parcial**: apenas uma fração da população é mantida, enquanto o restante é substituído. Serão testadas três variações de tamanho populacional (100, 200 e 300 indivíduos), três proporções de substituição (25%, 50% e 75%) e as mesmas combinações de taxa de mutação com torneios binários de tamanho $k = 2$ descritas anteriormente;
- **Elitismo**: os melhores indivíduos da geração anterior são preservados e incorporados à nova população, garantindo que soluções de alta qualidade não sejam descartadas. Serão avaliadas três variações de tamanho populacional (100, 200 e 300 indivíduos), três proporções de elitismo (10%, 20% e 30%) e as combinações de taxa de mutação com torneios binários de tamanho $k = 2$ utilizadas nas demais estratégias.

Tabela 1. Configurações experimentais por estratégia de controle populacional

Estratégia	Parâmetro	Valores testados
Substituição total	Tamanho da população	100, 200, 300
	Taxa de mutação	10%, 20%, 30%
	Tamanho do torneio (k)	2
Substituição parcial	Proporção substituída	25%, 50%, 75%
	Tamanho da população	100, 200, 300
	Taxa de mutação	10%, 20%, 30%
	Tamanho do torneio (k)	2
Elitismo	Proporção de elite	10%, 20%, 30%
	Tamanho da população	100, 200, 300
	Taxa de mutação	10%, 20%, 30%
	Tamanho do torneio (k)	2

3.2. Experimentos

Os experimentos serão conduzidos a partir de $n = 4$. Após cada conjunto de 100 execuções independentes, o valor de n é incrementado em uma unidade. O processo é encerrado quando a taxa de sucesso observada nas 100 execuções cair abaixo de 50%, indicando que o cenário se tornou demasiadamente difícil para a configuração avaliada. Esse procedimento garante que cada estratégia seja analisada dentro de uma faixa de complexidade progressiva, mantendo comparabilidade entre os experimentos e evitando limites artificiais definidos apenas pela disponibilidade computacional. Para cada variação experimental de controle populacional, são realizadas 100 execuções por valor de n, com o objetivo de reduzir o viés estocástico e aumentar a robustez estatística dos resultados.

Cada execução é encerrada quando uma solução sem conflitos é encontrada ou quando se atinge o número máximo de gerações. Durante as execuções, serão coletadas as seguintes métricas:

- **Tempo de execução**: tempo, em segundos, até o algoritmo encontrar uma solução ou atingir o número máximo de gerações;
- **Número de gerações**: quantidade de gerações necessárias para encontrar uma solução. Será adotado um limite máximo de 1.000 gerações, a fim de evitar execuções excessivamente longas em casos de não convergência;
- **Taxa de sucesso**: proporção de execuções que encontraram uma solução válida dentro do limite de gerações estabelecido.

Tabela 2. Métricas coletadas nos experimentos

Métrica	Descrição
Tempo de execução	Tempo necessário, em segundos, para encontrar uma solução válida ou atingir o limite de gerações estipulado.
Número de gerações	Quantidade de gerações necessárias até que uma solução válida seja encontrada.
Taxa de sucesso	Proporção de execuções que obtiveram solução válida dentro do limite de gerações estabelecido.

3.3. Codificação e Ambiente de Execução

A implementação será realizada em `Python`, apoiando-se em bibliotecas que já aparecem nos scripts do repositório: `numpy` (com o gerador `numpy.random.default_rng` para todas as operações estocásticas), `secrets` (para a criação de sementes imprevisíveis), `numba` (para acelerar os núcleos mais custosos, com retrocesso automático para `numpy` puro quando indisponível), `argparse` (para parametrizar as execuções pela linha de comando), `psutil` (na coleta opcional de métricas de hardware), `time` (para mensuração de duração) e `matplotlib` (para a síntese gráfica dos resultados). Esse conjunto reflete as dependências efetivamente utilizadas em `code/`, removendo a referência ao módulo `random`, ausente nos experimentos automatizados.

A geração de aleatoriedade é reproduzível: cada campanha registra a semente realmente empregada, inicializada com `secrets.randbits` quando não fornecida pelo

usuário, e todas as permutações, torneios e mutações são conduzidos com o gerador vetorizado da NumPy. As métricas de execução (tempo total, número de gerações, taxa de sucesso e contadores de operadores) são acumuladas durante as rodadas e exportadas em arquivos CSV e JSON para posterior análise.

Todos os testes são executados em um ambiente computacional controlado, com configurações fixas de hardware e software, garantindo reprodutibilidade conforme as boas práticas descritas por Eiben e Smith [Eiben and Smith 2015]. No início de cada campanha, o script de experimentos consulta o sistema operacional, modelo de CPU, quantidade de núcleos lógicos, memória física disponível e versões de Python, NumPy e Numba (quando presentes), recorrendo ao `psutil` sempre que disponível para complementar as medições. Esses metadados são anexados aos resultados exportados, permitindo reconstruir o contexto de execução.

3.4. Análise dos Resultados

Os dados obtidos a partir das execuções experimentais serão organizados e analisados estatisticamente com o apoio de medidas de tendência central e dispersão. Para cada combinação de experimento e valor de n, serão calculados média, desvio padrão e intervalos de confiança de 95% para tempo de execução e número de gerações, bem como o intervalo de confiança da taxa de sucesso a partir da aproximação de Wilson. Além dos resultados de desempenho, cada registro inclui metadados automáticos sobre o ambiente (hardware, sistema operacional e versões de bibliotecas) coletados no início da campanha, favorecendo a reprodutibilidade das análises. Esses indicadores fornecem um panorama mais completo do comportamento do AG ao longo das 100 execuções realizadas.

A comparação entre as diferentes estratégias de controle populacional será conduzida com testes estatísticos para verificar a significância das diferenças observadas. Para métricas contínuas, como tempo de execução e número de gerações, serão aplicados testes z com correção para amostras independentes, além do cálculo do tamanho de efeito de Cohen (d). Para as taxas de sucesso, será empregado um teste de diferença de proporções com a mesma distribuição de referência e o tamanho de efeito de Cohen (h). Esses recursos permitem avaliar não apenas a existência de diferenças, mas também sua magnitude prática.

Os resultados serão apresentados por meio de tabelas, gráficos com barras de erro representando os intervalos de confiança e relatórios consolidados com os valores de p e tamanhos de efeito. A análise buscará destacar tendências, padrões de comportamento entre os cenários experimentais e possíveis relações entre os parâmetros avaliados. Além disso, serão discutidas as implicações práticas das escolhas metodológicas adotadas, bem como suas contribuições para a compreensão do impacto das estratégias de controle populacional no desempenho de algoritmos genéticos.

4. Desenvolvimento

4.1. Implementação da Ferramenta Experimental

4.2. Execução dos Experimentos

Com a infraestrutura consolidada, o foco passa à execução sistemática dos cenários experimentais descritos na Seção 3. Para cada combinação de estratégia de controle populacional, tamanho de população, taxa de mutação, tamanho de torneio e valor de n,

a ferramenta realiza 100 execuções independentes registrando tempo de processamento, número de gerações e taxa de sucesso. Além disso, o ambiente de execução (sistema operacional, CPU, memória e versões das principais dependências) é capturado no início da campanha e anexado às observações exportadas. Os dados são armazenados em arquivos no formato `CSV`, facilitando a posterior agregação e análise estatística e a visualização por meio de gráficos comparativos.

4.3. Análise Parcial e Resultados Esperados

5. Conclusões

Referências

Ali, H. et al. (2023). Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem. In *Proceedings of ICIT*.

Bell, J. and Stevens, B. (2009). A survey of known results and research areas for n-queens. *Discrete Mathematics*, 309(1):1–31.

de Lacerda, E. G. M. and de Leon Ferreira de Carvalho, A. C. P. (1999). Introdução aos algoritmos genéticos. In *Sistemas inteligentes: aplicações a recursos hídricos e sistemas ambientais*. Ed. Universidade / UFRGS / ABRH, Porto Alegre.

Eiben, A. E. and Smith, J. E. (2015). *Introduction to Evolutionary Computing*. Natural Computing Series. Springer, Berlin Heidelberg, 2 edition.

Farhan, A. S., Tareq, W. Z., and Awad, F. H. (2015). Solving n queen problem using genetic algorithm. *International Journal of Computer Applications*, 122(12).

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Python Software Foundation (2025a). *The Python Language Reference, version 3.x*. Python Software Foundation. Acesso em: 28 jun. 2025.

Python Software Foundation (2025b). *Python Module Reference: random — generate pseudo-random numbers*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Python Software Foundation (2025c). *Python Module Reference: time — Time access and conversions*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Richter, H. and Berlezi, R. (2012). Algoritmo genético aplicado ao problema das n rainhas. *Anais da UNIJUÍ*. Relatório técnico disponível em proceedings.sbmac.

Sarkar, U. and Nag, S. (2017). An adaptive genetic algorithm for solving n-queens problem. *arXiv preprint*.

Spears, W. M. (1993). Crossover or mutation? In Whitley, D., editor, *Foundations of Genetic Algorithms 2*, pages 221–237. Morgan Kaufmann.